

操作系统期末考试复习

题型

- 选择题 3分一个 10个 **选择题可能会比较难**
 - 基本概念
 - 知识的综合运用
- 计算题 10分一个 4个
 - 计算
 - 分析
 - 设计
- 系统分析题 15分 2个
 - 综合设计
 - 分析
 - 优化

实验部分 10%-20%

各部分占比:

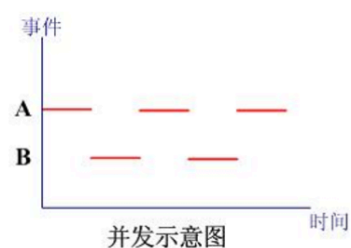
- **进程管理 30-45**
- **内存管理 30-45**
- 文件系统 10-15
- 设备管理 10-15
- 死锁 5-10
- 安全 不考

复习

进程管理

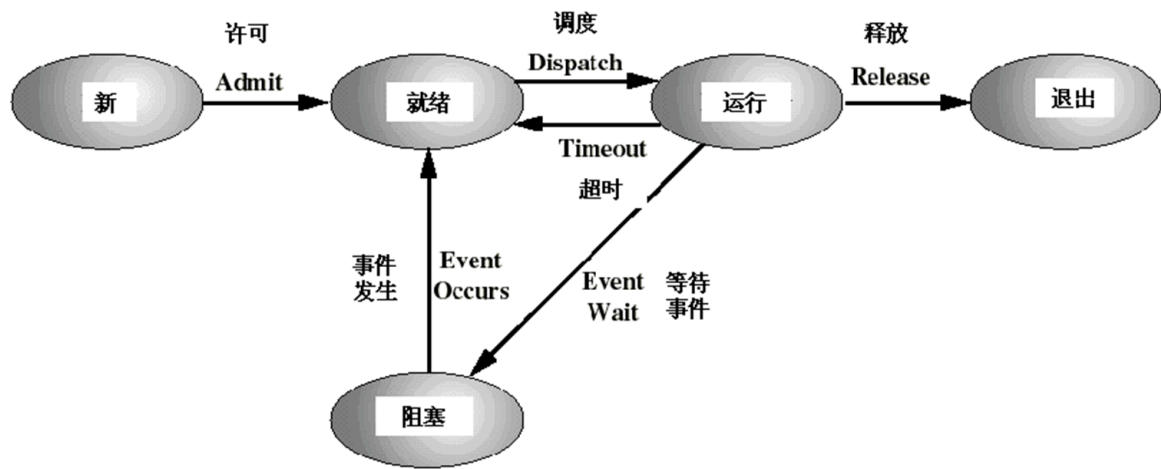
并发&并行

- 并行: 多个任务在同一时间同时运行
- 并发: 微观层面, 同一时刻只有一个任务在运行; 宏观层面, 多个任务在差不多相同的时间推进

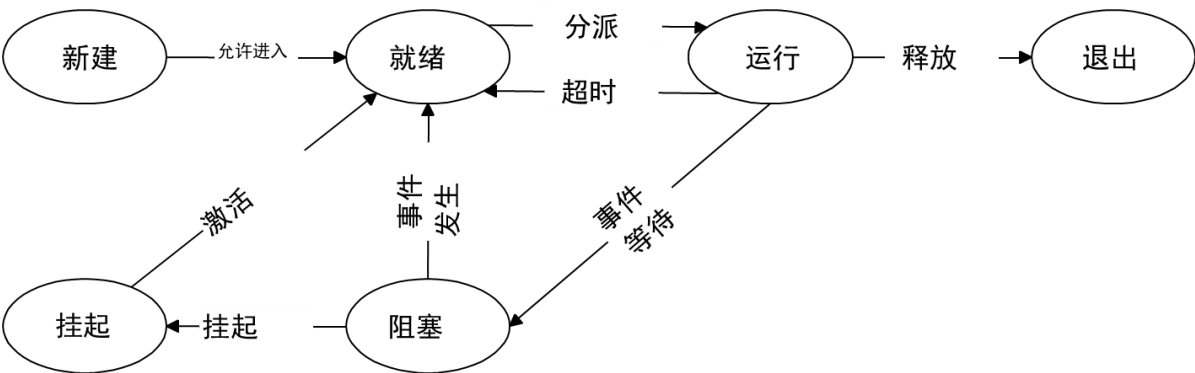


进程中的五状态模型

——要求把状态啥的都画下来，各状态的跳转与含义都要知道

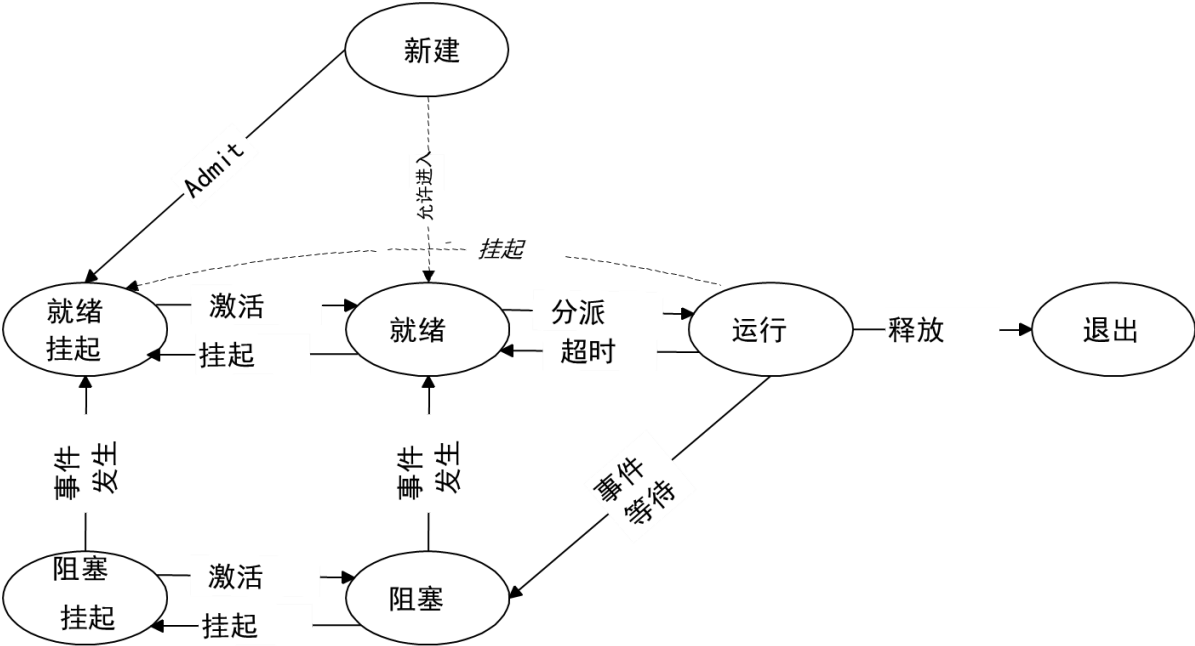


单挂起进程模型



加入内存资源的管理之后，挂起状态对于用户态的程序不可见
状态的跳转和含义都要知道

双挂起进程模型



进程调度算法

- 先来先服务——对于长任务的时间浪费很多，CPU利用率较低
- 短任务优先，剩余时间短任务优先——长任务会有很长的等待时间
- 时间片轮转
- 优先级队列

调度算法的设计原则：最优解，可以获取的参数，推理的代价

调度算法都要复习，不仅仅是这四个

什么时机进行调度？

- 进程主动调度，主动让出CPU
- 中断抢占调度

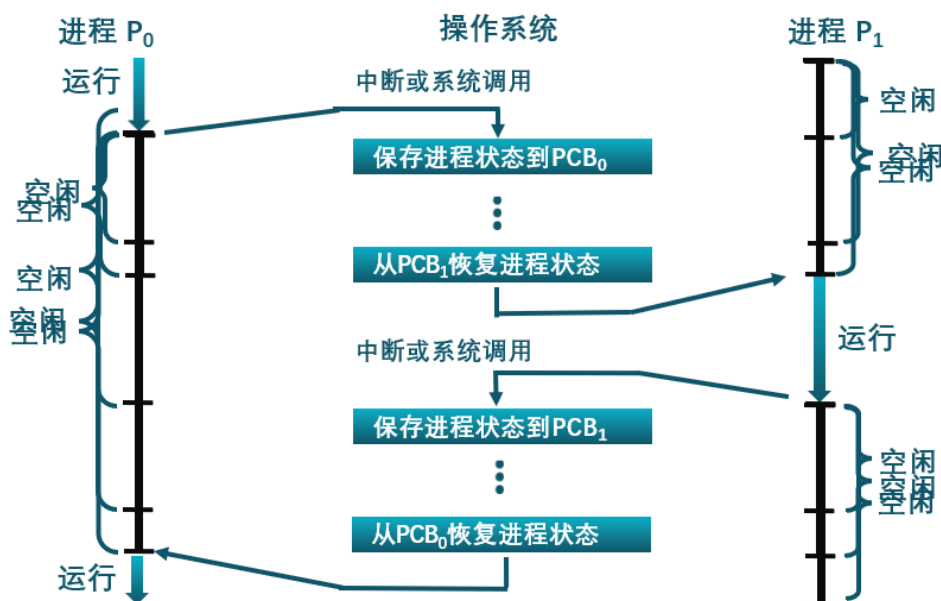
设计题——根据需求去设计对应的调度算法，合理即可

调度算法的几个重要的指标——CPU使用率、吞吐量、周转时间、等待时间、响应时间

进程切换

切换前，要保存哪些状态？跳转回来之后，怎么找到下面运行的代码？

上下文切换——重点，熟练掌握



复习讲到的案例

switch_to的实现，考察的时候，给出源代码，找到代码，哪几行是干了什么，为什么要干

上下文切换的代码需要仔细看

```

Terminal 终端 - xiaoli@xiaoli-VirtualBox: ~/ucore/os kernel lab/labcodes answer/lab8
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
return last_pid;

// proc_run - make process "proc" running on cpu
// NOTE: before call switch_to, should load base addr of "proc"'s new PDT
void proc_run(struct proc_struct *proc) {
    if (proc != current) {
        bool intr_flag;
        struct proc_struct *prev = current, *next = proc;
        local_intr_save(intr_flag);
        {
            current = proc;
            load_esp0(next->kstack + KSTACKSIZE);
            lcr3(next->cr3);
            switch_to(&(prev->context), &(next->context));
        }
        local_intr_restore(intr_flag);
    }
}

// forkret -- the first kernel entry point of a new thread/process
// NOTE: the addr of forkret is setted in copy_thread function
"kern/process/proc.c" 1115 行 --19%--
221,4

```

1. 在哪里换的虚拟地址空间? **cr3**
2. 换了空间为啥不影响 switch_to 的执行? **地址空间重叠**
3. TLB中存的是什么? **页表的部分信息**
4. 为什么换页表不用管TLB? **写入cr3时自动失效**
5. 这样有什么缺陷? **重复的页表项也没了。**
6. 有cache吗? Cache怎么处理? **丢弃, 或者用实地址cache**
7. 为什么换了地址空间不需要重新定位变量? 因为多个进程之间内核空间是共享和统一的

调度的代价

就是各类的优缺点

进程间通信

信号、管道、消息队列、共享内存——都要掌握

线程

用户空间线程、内核线程、混合式

线程应该在用户态还是内核态?

ULT管理优势:

- 线程切换不需要内核模式特权.
- 线程调用可以是应用程序级的,根据需要可改变调度算法,但不会影响底层的操作系统调度程序.
- ULT管理模式可以在任何操作系统中运行,不需要修改系统内核,线程库是提供应用的实用程序。

ULT的劣势:

- 系统调用会引起进程阻塞
- 这种线程不利于使用多处理器并行

核心级线程 (KLT): 线程由OS内核进行管理, 内核给应用程序级提供系统调用, 实现对线程的使用。

特点:

- 线程在内核中有保存的信息,系统调度是基于线程完成的.
- 可克服ULT的两个缺点,且内核程序本身也可以是多线程结构的.

劣势:

- 线程间的控制转换需要转换到内核模式。(为什么)

26

线程、协程 也要看一下

并发与同步

竞争、临界区, 共享数据

互斥&同步——空闲则入, 忙则等待, 有限等待

关中断——理论上可以解决单核心的问题, 但是为什么不能用? ——不安全

原语、信号量和管程 来实现并发和同步——各种锁的机制

经典问题——生产者消费者、沉睡的理发师、饥饿的哲学家、读者写者——都要写出代码, 还要扩展, 设计新的互斥和共享的效果

我们需要记住设计的套路——互斥就加锁, 同步也是加锁, 两种锁机制, 顺序不要错

用信号量实现临界区的伪码形式

每个临界区设置一个信号量，其初值为1

```
mutex = new Semaphore(1);
```

```
mutex->P();  
Critical Section;  
mutex->V();
```

- 必须**成对使用P()操作和V()操作**
 - ▣ P()操作保证互斥访问临界资源
 - ▣ V()操作在使用后释放临界资源
 - ▣ PV操作**不能次序错误、重复或遗漏**

这个很重要!!! 加PV锁，解决临界区的问题

用信号量实现条件同步

每个条件同步设置一个信号量，其初值为0

```
condition = new Semaphore(0);
```

线程A

```
... M ...  
condition->P();  
... N ...
```

线程B

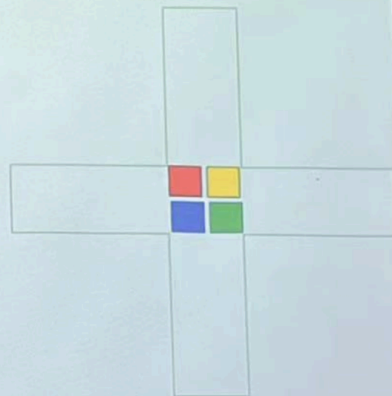
```
... X ...  
condition->V();  
... Y ...
```



例题：——死锁了

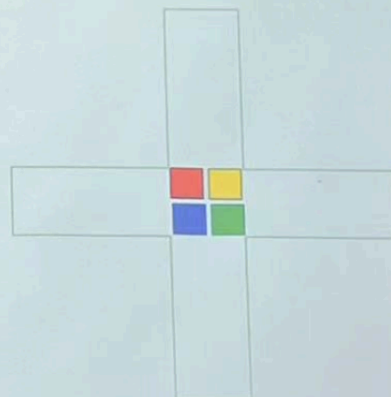
学会使用信号量设计问题的解决方案

- 如何设计信号量，可以使得车辆在通行时不会发生事故？
- 如何设计，可以使得不出现一方堵塞时间过长的情况？
- 如何设计可以避免死锁？



解析

- 申请四个信号量R,Y,B,G分别对应四块路面
- 以从左到右通行的车辆为例
 - $P(B)$, $P(G)$ 成功后，就可以通行，通行离开后，释放资源即可
- 通过观察不难发现，最极端的情况，也只能有3辆车同时通过，所以可以再设置一个进入路口用的信号量，不要让4辆车同时到达路口



例题做一下：

再一个例题

11. 假设有一个数据类型 `complex` 定义如下:

(1) 什么是互斥? 这段代码里哪些操作需要互斥? (2分)

(2) 下图给出了3个线程对应的代码, 假设线程的临时变量都放在栈区, 请定义相应的信号量, 用PV操作保证代码的正确性, 并尽可能减少线程间不必要的等待。(8分)

Complex 类型的代码示意	thead1	thead2	thead3
<pre> class Complex { protected: double real; double img; public: friend Complex add(Complex p, Complex q) { Complex s; s.real=p.real+q.real; s.img=p.img+q.img; return s; } //其他代码省略 }; Complex x, y, z; //共享的全局变量 </pre>	<pre> Threadfunc1 { Complex w; //无关代码 w=add(x,y); //无关代码 } </pre>	<pre> Threadfunc2 { Complex n; //无关代码 n=add(y,z); //无关代码 } </pre>	<pre> Threadfunc3 { Complex w; //无关代码 z=add(z,w); y=add(y,w); w=add(x,z); //无关代码 } </pre>

使用锁 来解决数据冲突问题

多核SMP结构下指令可以乱序执行引发的新问题

考虑如下代码:

CPU0	CPU1		CPU0	CPU1
a = 1;	while (true) {		a = 1;	while (true) {
b = 2;	if (flag)	乱序执行	b = 2;	if (flag)
c = 3;	do_something(a, b, c);	→	flag = true;	do_something(a, b, c);
flag = true;	}		c = 3;	}

如果CPU0顺序执行, a、b、c初始化完毕后CPU1再do something, 符合预期。

当CPU0乱序执行, c还未初始化CPU1就do something, 出错。

Peterson算法和Deckers算法能够在理论上完成单核正确性的证明, 但是在多核运行时, 由于指令执行顺序交换而导致原来的依赖性问题无法保障, 因此需要内存屏障指令才能保证其正确性

Peterson算法、Deckers算法都要掌握!!!

系统调用和保护机制——设置不同的权限

系统调用库的接口需要看一下代码!

- 哪一个指令提升了权限？ **int**
- 为什么提升了权限不会破坏安全性？
因为权限提升之后只会执行固定的代码，而这段代码一定会回收权限
- 怎么知道要干什么？
编号和参数是用户和OS的约定
- 为什么syscall的代价大？因为要有复杂的参数检查、安全性检查，导致运行了一些无用的代码，以及硬件上的清理工作

系统调用库接口示例

```

sfs_filetest1.c: ret=read(fd,data,len);
.....
8029a1: 8b 45 10      mov 0x10(%ebp),%eax
8029a4: 89 44 24 08    mov %eax,0x8(%esp)
8029a8: 8b 45 0c      mov 0xc(%ebp),%eax
8029ab: 89 44 24 04    mov %eax,0x4(%esp)
8029af: 8b 45 08      mov 0x8(%ebp),%eax
8029b2: 89 04 24      mov %eax,(%esp) ; 以上代码在填充栈
8029b5: e8 33 d8 ff ff call 8001ed <read> ; 这里的read是宏==6
syscall(int num, ...) {
...
asm volatile (
    "int %1; /*这一句会产生一个中断*/
    : "=a" (ret)
    : "i" (T_SYSCALL), /*这句指定中断号*/
      "a" (num), /*这一句的作用是把6放进了eax*/
      "d" (a[0]),
      "c" (a[1]),
      "b" (a[2]),
      "D" (a[3]),
      "S" (a[4])
    : "cc", "memory");

return ret;

```

```

// System call numbers
#define SYS_fork 1
#define SYS_exit 2
#define SYS_wait 3
#define SYS_pipe 4
#define SYS_write 5
#define SYS_read 6
#define SYS_close 7
#define SYS_kill 8
#define SYS_exec 9
#define SYS_open 10
#define SYS_mknod 11
#define SYS_unlink 12
#define SYS_fstat 13
#define SYS_link 14
#define SYS_mkdir 15
#define SYS_chdir 16
#define SYS_dup 17
#define SYS_getpid 18
#define SYS_sbrk 19
#define SYS_sleep 20
#define SYS_procmem 21

```

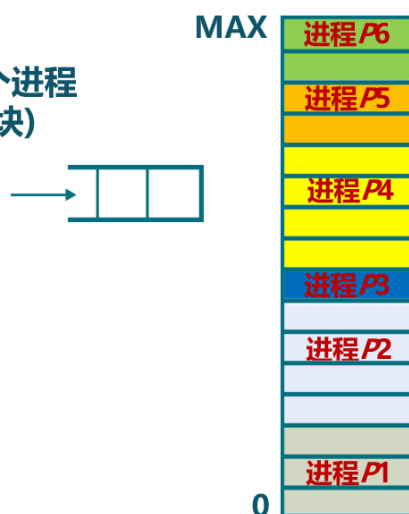
系统调用read函数的实现——

内存管理

内存分配

几种内存分配的算法都要知道

- 动态分区分配
 - 当程序被加载执行时，分配一个进程指定大小可变的分区(块、内存块)
 - 分区的地址是连续的
- 操作系统需要维护的数据结构
 - 所有进程的已分配分区
 - 空闲分区(Empty-blocks)



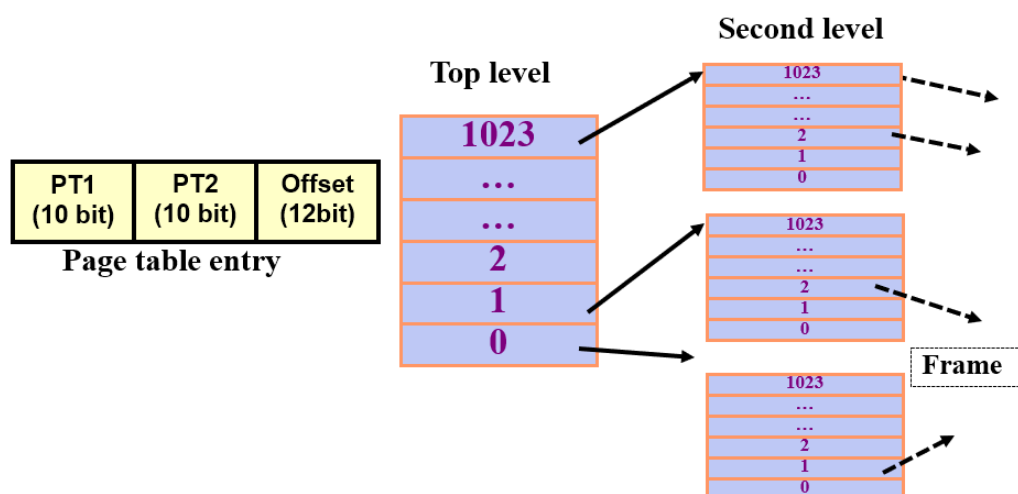
内存分配和回收之后，对于内存碎片的处理

如果空间还是不够，就要使用虚拟内存的方案：

覆盖式、分页式、分段式、段页结合...

页表&多级页表

页表的设计能力



混合的页表结构要会设计

4K大小的页一定需要是4K吗？不一定

64位系统的页表，自主设计

X64的页表是什么样的

- 每个页表里面有512项
 - 4k/8字节(64bit) = 512
 - 因此用于页表项的寻址部分仅有9位
- 类似的，页目录项也只有512项
 - 页目录项也只有9位
- 页表扩展为4级，每级都是9位地址
 - 寻址空间最大为256T
 - 最高的16位目前没有使用

感觉会考（

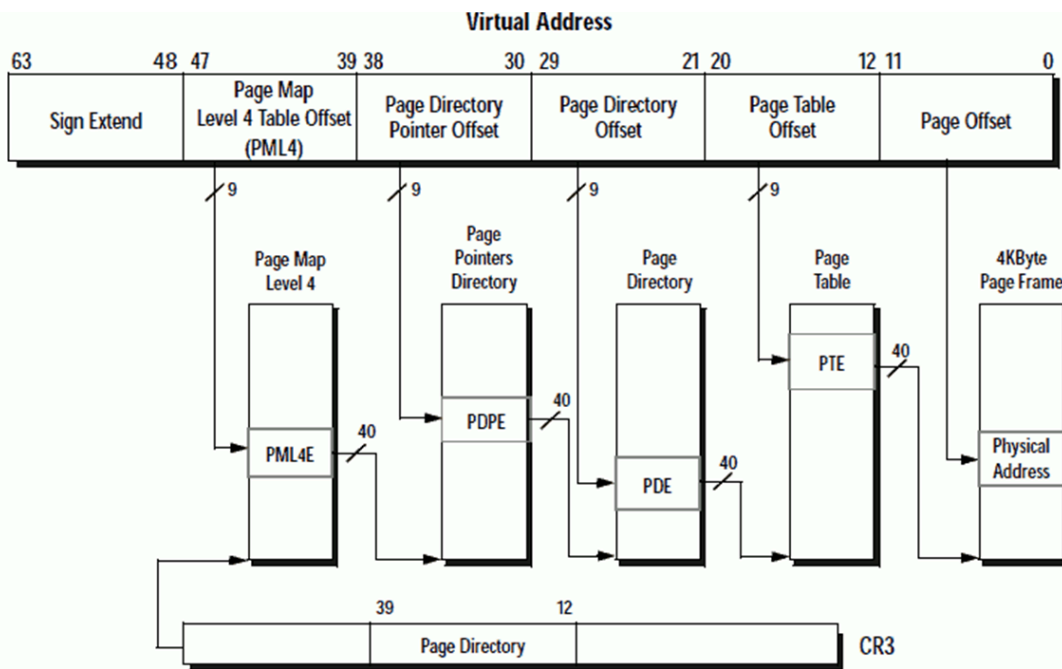
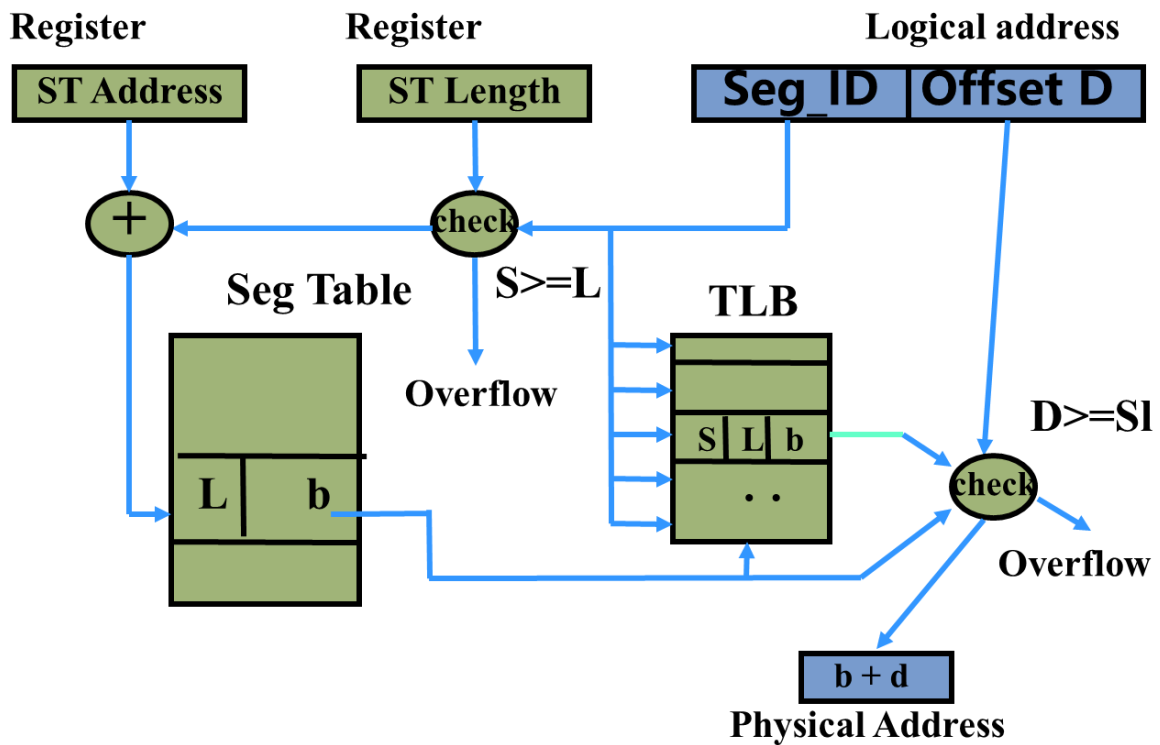


Figure 17. 4KB-Page Translation

TLB快表

——过程分析，原理分析



页面的大小一定是4K吗？

4K的页真的是不可动摇的吗：回忆ucore

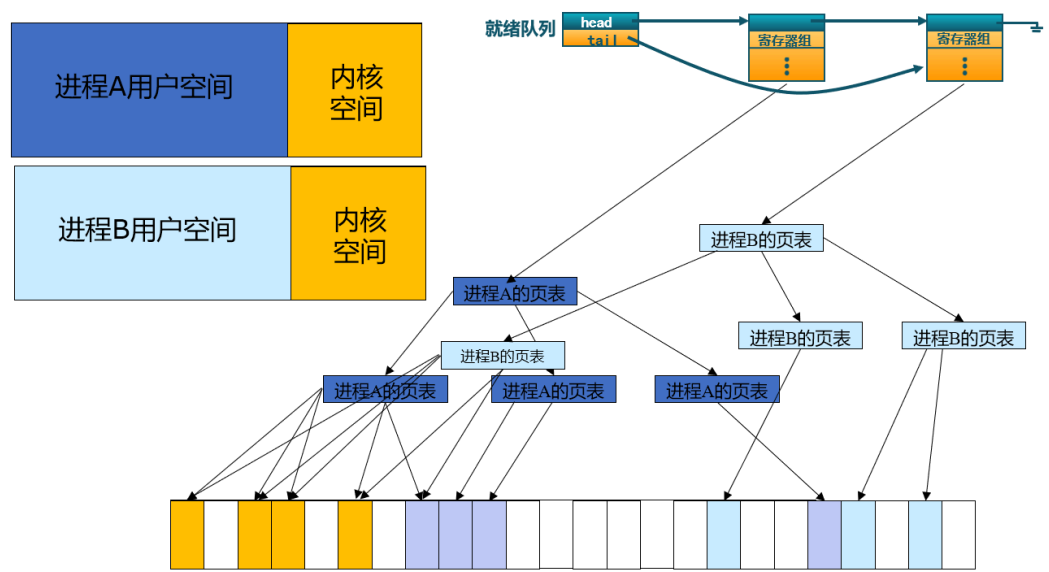
- 4K的页
- 2M的页
- 1G的页
- 大页有什么优缺点？
 - 省页表的查找时间
 - 省TLB项
 - 浪费内存（页内效率低，页间可能有碎片）
 - 破坏兼容性

```
graph TD; SATP[SATP] --> E1[512项]; SATP --> P1[2M页]; SATP --> P2[1G页]; E1 --> E2[512项]; E1 --> P3[4K页]; E2 --> P4[2M页];
```

可以设计不是4K大小的页表，去年就考了这个（
优缺点都要掌握

fork&exec函数

fork调用后,exec调用， 执行了一段时间后

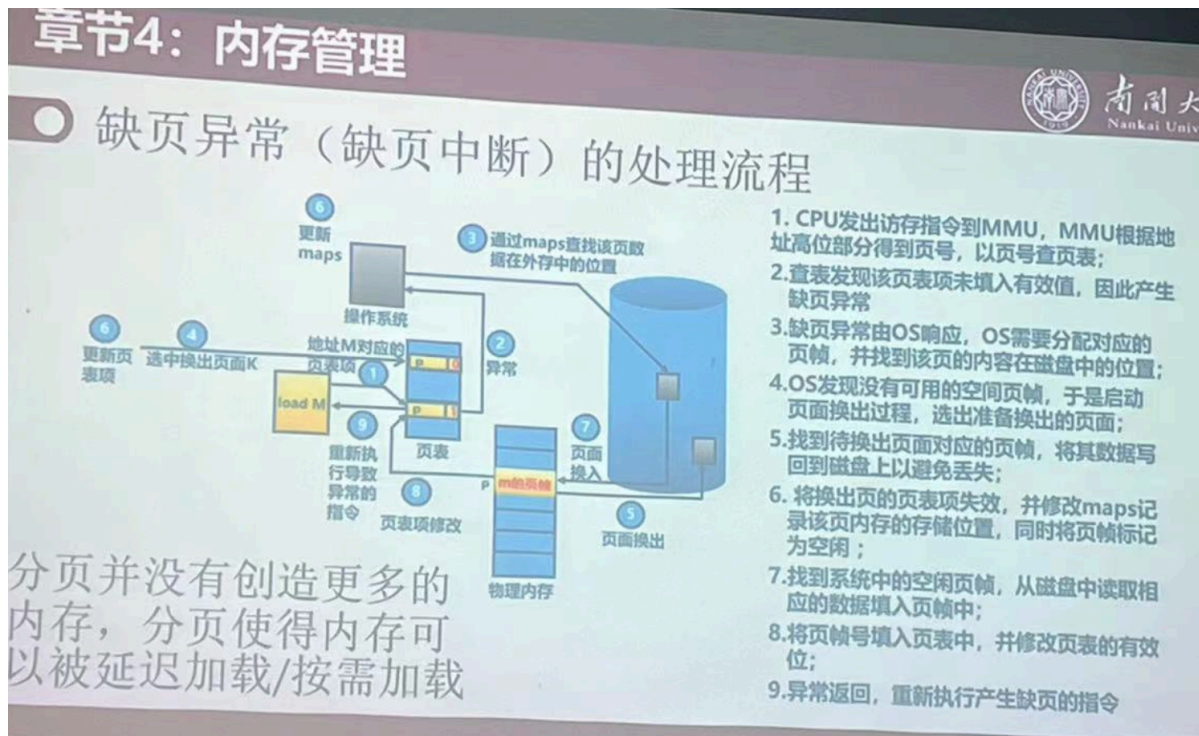


缺页中断

缺页中断的流程——重要

综上所述，缺页异常的全过程：

- 编译阶段把虚拟地址空间划分成了若干区域
- 链接阶段将这些区域信息和相应的数据和指令存在文件中
- 加载阶段OS读取这些区域信息和存储的对应关系（并不加载真正的数据和指令）
- 程序访问某个区域，触发缺页异常，在异常的响应函数中，完成对数据和指令的加载，并出现在物理内存和页表中
- 缺页异常的响应函数返回至触发异常的指令，该指令得以再次执行，并且不再触发异常



页面置换算法

列表计算，都需要掌握！！

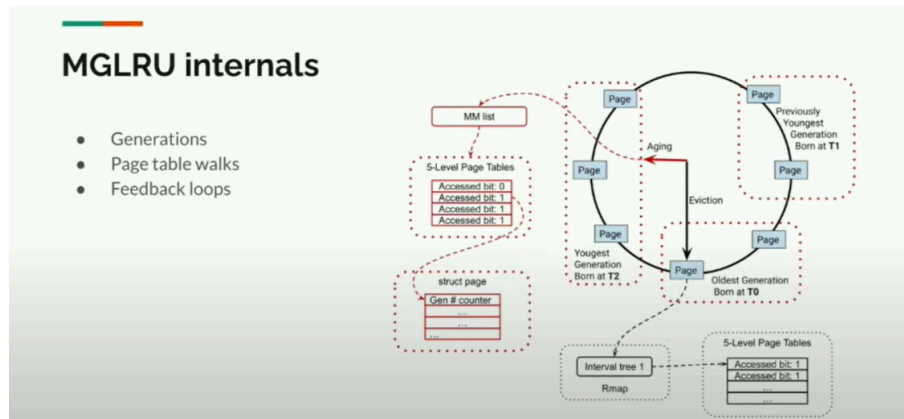
Summary: Page replacement algorithm

	Principle	Performance
最优方法	预知未来情况的上限	永远不可能达到
FIFO	按时间排序，最简单	来得最早，和最不可能用到之间没有关系
LRU & LFU	利用过去使用的频度和次数预测未来	目前可达的最佳效果，代价略高
CLOCK	FIFO与LFU的折衷	足够简单，也有一定的合理性，在OS中大量使用
Working set	只给进程需要的内存	不现实
Page Fault Frequency	根据缺页事情的频度预测内存的需求	相对现实

计算缺页率，要会！画出全部的图

真实的操作系统，比如linux使用的换页算法使用了MGLRU

补充: linux内核中的换页机制



我们把page分为很多种类，比如youngest和oldest的page，我们根据每一轮的时钟读取，如果一轮中没有读到这个页面，我们就把这个页面的等级做一个改变。每当出现了缺页的情况时，我们就把那堆oldest的页表存储到硬盘中即可

设计自己的内存页面调度机制，也要会

belady现象

增加页面之后，发现不能降低我们的缺页率

本质是什么？掌握！

tips: 对于一次访存而言，LRU不会出现belady是因为，增加页框数量后原本不引起缺页事件的访存必然不会引起缺页

- 因为留在页框中的内容是按最近访问的顺序排列的，增加页框，得到的留在内存中的页面，一定是增加之前页面集合的超集(也就是说，如果我们原来有3个页，那么3个页中存储的一定是最近访问的三个页；那么我们将页数增加到4的时候，我们4个页中存储的一定是最近访问的四个页，这四个页一定包含了前面的三个页，所以不会出现belady现象)
- 新增的页框最差情况下就是不发挥作用，但是不会引出新的缺页事件，因此不会增加

LRU算法的缺陷

抖动问题

文件系统

文件的管理

链表型&树型

文件夹的管理

文件数据的一致性

VFS

用户权限

link table

index table

存储体系结构的使用

计算机世界中的时间——读取内存十分耗费时间

磁盘的访问调度的策略——三种算法，都要掌握！！！！

这个很重要，感觉考个计算？

磁盘的缓存

设备管理

IO资源的管理

独立编址，共享编址，混合式

三种设备的通信模式

忙等待，中断，DMA

驱动程序设计中的典型问题

异步，缓冲，spooling

硬盘管理中的若干问题

RAID，磁臂管理

死锁

四种方案

方法	资源分配策略	各种可能模式	主要优点	主要缺点
预防 Prevention	保守的；宁可资源闲置（从机制上使死锁条件不成立）	一次请求所有资源<条件 1>	适用于作突发式处理的进程；不必剥夺	效率低；进程初始化时间延长
		资源剥夺<条件 3>	适用于状态可以保存和恢复的资源	剥夺次数过多；多次对资源重新启动
		资源按序申请<条件 4>	可以在编译时（而不必在运行时）就进行检查	不便灵活申请新资源
避免 Avoidance	是“预防”和“检测”的折衷（在运行时判断是否可能死锁）	寻找可能的安全的运行顺序	不必进行剥夺	使用条件：必须知道将来的资源需求；进程可能会长时间阻塞
检测 Detection	宽松的；只要允许，就分配资源	定期检查死锁是否已经发生	不延长进程初始化时间；允许对死锁进行现场处理	通过剥夺解除死锁，造成损失
忽略 ignore	不理睬死锁问题，认为它不是主要问题（鸵鸟法）			

银行家算法

——掌握！！